User Authentication and Application Integration

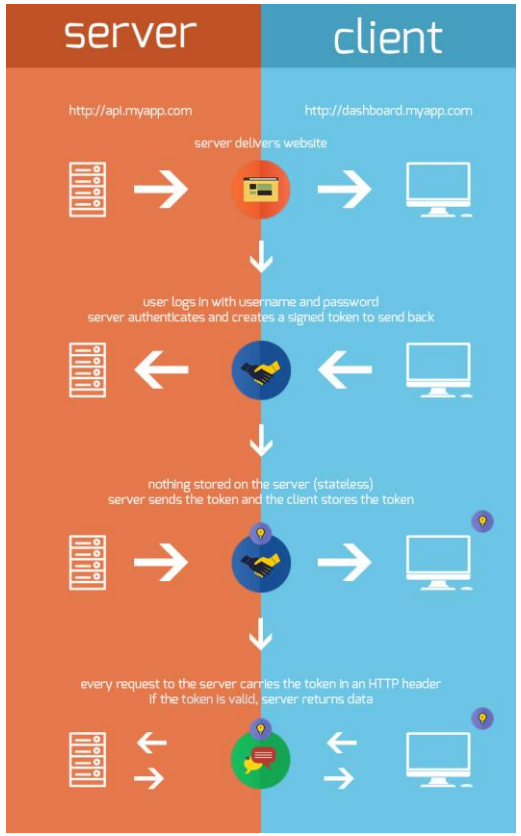
Secured login and better user experience

Outline

- Login and registration systems
- Password hashing
- Access control
- UI/UX integration

Objective

- Understand the concept of user authentication
- Implement login and registration systems in PHP
- Apply password hashing for security
- Implement access control mechanisms
- Integrate authentication with UI/UX



Introduction

- User authentication integrates login/registration with PHP and MySQLi for secure app access.
- It covers password protection, role checks, and seamless UI embedding.
- This ensures data privacy in dynamic web apps.

User Authentication

- User authentication is the process of verifying the identity of a user before allowing access to a system.
- Common methods:
  - ✓ Username and password
  - ✓ Two-factor authentication
  - ✓ Biometrics
- In web systems, authentication usually involves verifying user credentials stored in a database.

Application Integration

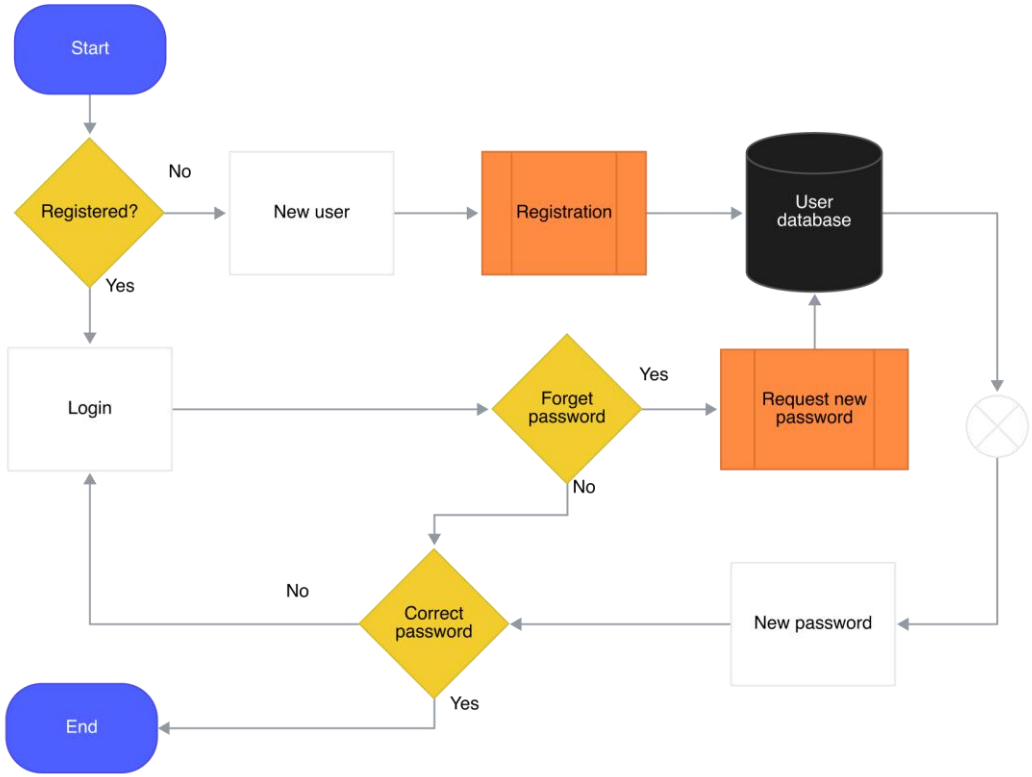
- Application integration connects different components of a system to work together.

- In PHP web apps, authentication integrates with:
  - ✓ Database (MySQL)
  - ✓ User Interface (HTML/CSS)
  - ✓ Session management
  - ✓ Access control mechanisms

Goal: Seamless and secure user experience.

Login and Registration Systems

- Registration System:
  - Allows users to create accounts
  - Stores user data in a database
- Login System:
  - Verifies user credentials
  - Creates a session if authentication is successful
- Typical workflow:
  1. User submits form
  2. PHP processes request
  3. MySQL database validates or stores data



PHP MySQLi Registration Example

```
<?php
$conn = mysqli_connect('localhost','root','','mydb');

$username = $_POST['username'];
$email = $_POST['email'];
$password = password_hash($_POST['password'], PASSWORD_DEFAULT);

$sql = "INSERT INTO users (username,email,password)
VALUES ('$username','$email','$password')";

mysqli_query($conn, $sql);
?>
```

The Login System

A login system verifies credentials against MySQLi using sessions. PHP checks hashed passwords and starts sessions on success.

```
<?php
session_start();
$conn = mysqli_connect('localhost','root','','mydb');

$username = $_POST['username'];
$password = $_POST['password'];

$result = mysqli_query($conn,"SELECT * FROM users WHERE username='$username'");
$user = mysqli_fetch_assoc($result);

if($user && password_verify($password,$user['password'])){
  $_SESSION['user'] = $user['username'];
  echo 'Login successful’;
 } else {
  echo 'Invalid credentials';
}
?>
```

Password Hashing

- Password hashing converts a password into an irreversible string.
- Why it is important:
  - Protects passwords if the database is compromised
  - Prevents storing plain-text passwords
- PHP functions:
  - password_hash()
  - password_verify()
- These functions automatically handle salting and secure algorithms.
- Never store plain text or use MD5

Access Control

- Access control restricts what users can do after logging in.
- Examples:
  - Administrator vs Regular User
  - Protected pages
- Typical implementation:
  - Sessions
  - Role checking
  - Redirecting unauthorized users

Access Control Sample Code

```
session_start();
if(!isset($_SESSION['user']) || $_SESSION['user']['role'] !== 'admin’) {
header('Location: login.php’);
exit;
}
```

```
session_start();
if(!isset($_SESSION['user'])){
  header('Location: login.php');
  exit();
}
echo 'Welcome to the dashboard';
```

UI/UX Integration

- Authentication should provide a smooth user experience.
- **UI (User Interface)** is the visual part of a system that users interact with.
- **UX (User Experience)** refers to the overall experience a user has while interacting with a system. It focuses on how easy, efficient, and satisfying the system is to use.
- Good UI/UX practices:
  - Clear login and registration forms
  - Helpful error messages
  - Password visibility toggle
  - Responsive design
  - Redirect users after login
- UI usually built with HTML, CSS, and JavaScript while PHP handles the backend.

UI/UX Integration

User Interface	User Experience
Examples of UI elements ○ Buttons (Login, Register) ○ Input fields (Username, Password) ○ Menus and navigation bars ○ Colors, fonts, and layout ○ Icons and images	**Good UX characteristics** ○ Easy to understand ○ Fast and responsive ○ Clear error messages ○ Logical navigation ○ Minimal user frustration
Visual design of the system	Overall user experience
Buttons, forms, layout	Ease of use and satisfaction
What the user sees	How the user feels while using it

Summary

- Authentication verifies user identity
- Login and registration systems manage user accounts
- Password hashing protects sensitive credentials
- Access control restricts system resources
- UI/UX integration improves user experience
- Combining PHP, MySQL, and proper security practices creates reliable web applications.

User Authentication and Application Integration

Secured login and better user experience